

chapter - 08 Deadlocks

Deadlock is a situation in which every process in a set of processes is waiting for an event that can be caused only by another process in the set.

4 System Model: A system consists of a finite number of resources to be distributed among a number of competing threads.

Ex: CPU cycles, files, I/O devices are examples of resources types.

⇒ A thread may utilize a resource in only the following sequence:

1. Request. The thread request the resource. If the request can't be granted immediately then the requesting thread must wait until it can acquire the resource.
2. Use. The thread can operate on the resource.
3. Release. The thread releases the resource.

Deadlock is possible if thread-one acquires first-mutex while thread-two acquires second-mutex.

Ex: Data:

→ A semaphore s_1 initialized to 1

→ A semaphore s_2 initialized to 1

Two processes P_1 and P_2

P_1 :

wait(s_1)

wait(s_2)

P_2 :

wait(s_2)

wait(s_1)

Livelock ⇒ Livelock is another form of liveness failure.
It is similar to deadlock.

Deadlock Characterization:

Deadlock can arise if four conditions hold simultaneously:

1. Mutual exclusion: Only one process at a time can use a resource.
2. Hold and wait: a process holding at least one resource is waiting to acquire additional resources held by other processes.
3. No preemption: A resource can be released only voluntarily by the process holding it, after that process has completed its task.
4. Circular wait: A set $\{T_0, T_1, \dots, T_n\}$ of waiting threads must exist such that T_0 is waiting for a resource held by T_1 , T_1 is waiting for a resource held by T_2 , ..., T_{n-1} is waiting for a resource held by T_n , and T_n is waiting for a resources held by T_0 .

b) Resource-Allocation Graph:

Deadlocks can be described more precisely in terms of a directed graph called a system resource-allocation graph.

This graph consists of a set of vertices V and set of edges E .

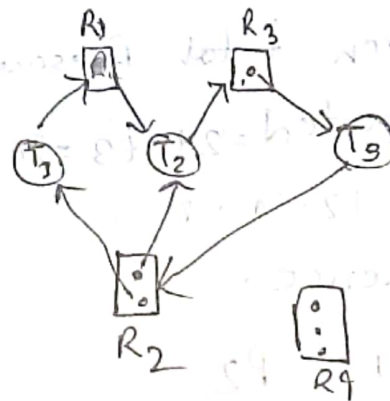
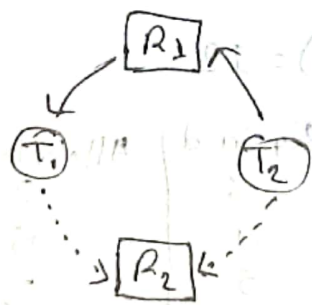
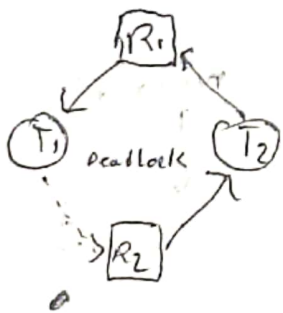
→ V is partitioned into two types:

- Resources (R) = $\{R_1, R_2, \dots, R_m\}$
- Process (P) = $\{P_1, P_2, \dots, P_n\}$

→ E (edge) types:

- Request edge ($P \rightarrow R$): A process is requesting/waiting for a resource.
- Allocated/Assignment edge ($R \rightarrow P$): A resource is currently allocated to a process.
- Claim edge ($P \rightarrow R$) (dashed line): A process may request this resource in the future.

Example:



Basic fact:

If graph contain no cycle $\Rightarrow$ no deadlock

If graph contain a cycle $\Rightarrow$

- If only one instance per resource type, then deadlock
- If several instances per resource type, possibility of deadlock.

- ⇒ Ensure that the system will never enter a deadlock state
 - Deadlock prevention
 - Deadlock avoidance

⇒ **Deadlock Prevention:**
 Invalidate one of the four necessary conditions for deadlock:

- Mutual exclusion, Hold and wait, No preemption
 - Circular wait.
- ≡ Inced this p

⇒ **Deadlock Avoidance:** Require that the system has some additional a priori information.

Safe state: If the system can successfully allocate resources to each process by giving resources, then it is called a safe state.

Ex: Given total Resource $(R_i) = 12$

Total allocated = $2 + 5 + 3 = 10$

Available = $12 - 10 = 2$

Safe sequence:

$P_3 \rightarrow P_1 \rightarrow P_2$

Process	Required	Allocated	Needed
P_1	6	2	4
P_2	11	5	6
P_3	5	3	2

Safe state find out which process now work.

Basic facts

- If a system is in a safe state $\Rightarrow$ No deadlocks.
- If a system is in unsafe state $\Rightarrow$ possibility of deadlocks
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.



2 Avoidance Algorithms:

- Single instance of a resource type: use a resource-allocation graph.
- Multiple instance of a resource type: use a Banker's algorithm

Resource Allocation Graph (RAG)

A RAG represent how processes and resources interact in a system. It helps analyze situations like deadlock.

* Same as before.

Banker's Algorithm:

- Multiple instance of resources.
- Each process must a priori claim maximum use.
- When a process request a resource it may have to wait.
- When a process gets all its resources it must return them in a finite amount of time.

Let n = number of process ; m = number of resources type

Available : Vector of length m . If $available[j] = k$, there are k instance of resources type R_j available.

max : $n \times m$ matrix. $\longrightarrow Max[i, j] = k$

Allocation : $n \times m$ matrix. $\longrightarrow Allocation[i, j] = k$

Need : $n \times m$ matrix. $\longrightarrow Need[i, j] = k$

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$

44) Safety ~~Allocation~~ Algorithm :

1. Let work and finish be vectors of length m and n respectively. Initialize

work = available

finish[i] = false for $i = 0, 1, 2, \dots, n-1$

2. Find an i such that both:

(a) finish[i] = false

(b) Need_i ≤ work.

if no such i exists, go to step 4

3. Work = work + Allocation_i

finish[i] = true

go to step 2.

4. If finish[i] = true for all i, then the system is in a safe state.

Here, R₁ = 10, R₂ = 5, R₃ = 7

Deadlock Avoidance

Process	Max Need (R ₁ , R ₂ , R ₃)	Allocated Resources (R ₁ , R ₂ , R ₃)	Current Resources Need (R ₁ , R ₂ , R ₃)	Available Re (R ₁ , R ₂ , R ₃)
P ₀	7 5 3	0 1 0 (iv)	7 4 3 (iv)	(iii) 3 3 2 (i)
P ₁	3 2 2	2 0 0 (i)	1 2 2 (i)	5 3 2 (ii)
P ₂	3 0 2	3 0 2 (iv)	6 0 0 (iv)	7 4 3 (iii)
P ₃	2 2 2	2 1 1 (ii)	0 1 1 (ii)	7 4 5 (v)
P ₄	4 3 3	0 0 2 (v)	4 3 1 (iii)	7 5 5
		7 2 5		10 5 7

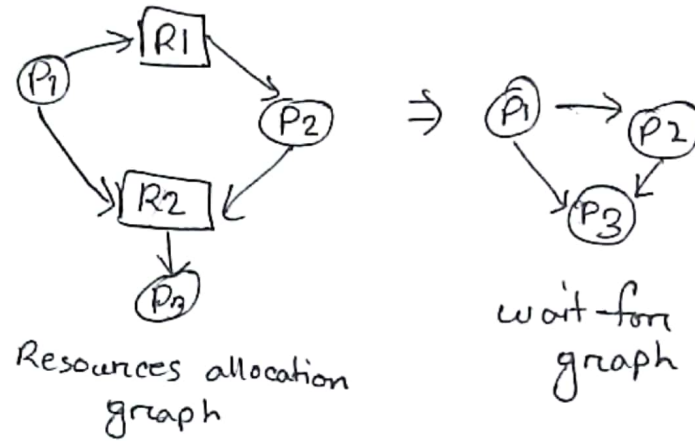
current Resource Need = Max Need - Allocated Resource

Safe sequence: P₁ → P₃ → P₄ → P₀ → P₂

Deadlock Detection:

- Allow system to enter deadlock state
 - Detection algorithm
 - Recovery scheme
- ← wait for graph
Banker's algorithm

① wait for graph:



Recovery from Deadlock:

1. Process termination
 - Abort all deadlocked processes
 - Abort one process at a time until the deadlock cycle is eliminated.
2. Resource Preemption
 - selecting a victim - minimize cost
 - Rollback - return to some safe state, restart process for that state.
 - Starvation - same process may always be picked as victim, include number of rollbacks in cost factor.